Scythe Apertus: Sovereign Autonomous Software Repair

Hack Apertus 2026 · Track 2B: Own Project · Team Syrava

Code: github.com/kapasainitishreddy/scythe-apertus-2026 **Demo:** 109-second fixture-based prototype preview. It does not show genuine Apertus inference.

Abstract and problem

AI-generated patches can look plausible while failing to repair the reported behavior. A repair tool also needs clear boundaries around source custody, permissions, and who decides whether a patch passes. Scythe Apertus is a small open-source prototype in which Apertus v1.5 diagnoses a defect and proposes a complete replacement for one allowlisted file. A separate offline worker runs independent assertions, and bounded test feedback can guide a second patch. The application records model responses, patch and test-output hashes, and the final verification decision. The benchmark contains twelve original JavaScript defects.

Why Apertus is central

Apertus v1.5 performs the task's four model-dependent steps: diagnosis, patch generation, feedback-guided retry, and a final critique. The controller sends OpenAI-compatible chat-completion requests to a configured endpoint. Model output must parse as JSON, pass a strict `bug.cjs` allowlist and size checks, and then pass the independent test worker. A model's confidence or critique cannot turn a failing test into a success. The model identifier is recorded, but the endpoint does not cryptographically attest which weights served a request.

Architecture and sovereign deployment

The Docker topology has an HTTP controller, a verifier worker, and an optional Apertus model server. The controller sends only a curated case's issue and broken source to the model. It transfers a validated candidate to the worker through a file queue. The worker runs the candidate in a separate Node process and returns a bounded result. The controller stores a JSON evidence record in a persistent Docker volume.

The intended sovereign deployment runs all three services on hardware administered by the organization. The official Swiss AI Initiative vLLM image is referenced by the optional Compose overlay; the model server is reachable only on the internal Compose network. The verifier has no network, receives no model credentials, mounts requests read-only, and runs with a read-only image filesystem, dropped Linux capabilities, no-new-privileges, resource limits, and Node's permission model. A hosted endpoint can be used for development, but it does not demonstrate sovereign operation.

Dataset, licensing, and prior work

The twelve cases cover pagination, hostname matching, retry limits, configuration values, deduplication, exact lookup, numeric bounds, date windows, cache expiration, sorting, integer parsing, and tax order. Each record contains a broken example, a reference repair, hidden assertions, provenance, and license metadata. The examples are original, synthetic, and contain no personal data. The dataset is CDLA-Permissive-2.0; source is Apache-2.0; authored documentation and media are CC BY 4.0. Apertus weights are not redistributed.

The broader Scythe/BlackScythe Forge IDE predates the hackathon and remains a separate private project. This public repository contains a separately packaged prototype and does not include that private source. The repair loop and benchmark implementation in this repository are the hackathon work described here.

Experimental method

The fixed evaluator first verifies that each broken fixture fails and its reference repair passes. The reference and hidden tests are not included in model prompts. A real run records the case, configured model and endpoint, diagnosis, every proposed patch, independent outcomes, retry count, wall time, token usage when returned, error category, and SHA-256 digest. Both JSON and TSV batch reports are produced. The evaluation uses bounded retries and does not change the verifier between cases. Since the fixtures and assertions are public, results are vulnerable to memorization and do not establish performance on repository-scale tasks. This custom set is not comparable to SWE-bench or its variants.

Results and failures

Check	Observed result on October 15, 2026
Broken fixtures failing independent assertions	12 / 12
Reference repairs passing those assertions	12 / 12
Local Node tests, including mocked HTTP integration and security cases	27 / 27
Genuine Apertus inference	Not run: no endpoint or credentials configured
Real-model repair rate, latency, and token cost	Not measured
Docker Compose build, start, and restart	Not run: Docker is unavailable on this host

The integration test uses a local mock completion endpoint and demonstrates the controller-to-worker flow only. It is not a model result. Local syntax checks passed. Docker Compose configuration was inspected in source, but `docker compose config`, image builds, and container startup could not be run. The official Track 2B `make run` command therefore remains **NOT VERIFIED** from a clean checkout.

Security and limitations

The model can propose source but cannot call a shell or apply arbitrary paths. The candidate runs only in the verifier process. Node permissions deny child-process creation, filesystem writes, and reads outside the temporary fixture. Docker's `network_mode: none` is the worker's network boundary; local host tests do not establish network isolation. The current verifier writes candidate and test files into the same temporary directory, so malicious candidate code could inspect public test source. The data is synthetic and public, the system supports one file and JavaScript only, and Docker is defense in depth rather than a formal containment proof. The dashboard is not designed for exposure to untrusted users.

Reproduction

From the repository root, run `make run` on a Docker-enabled machine to build and start the dashboard and offline worker. Open `http://localhost:8787` and use **Verify fixture** to check broken/reference discrimination without a model. For genuine model work, first accept the official Apertus model terms, serve Apertus v1.5 with the published Swiss AI vLLM image, and set `LLM_NAME`, `LLM_BASE_URL`, and any required `LLM_API_KEY` outside version control. On a GPU host, `make run-onprem` uses the included serving overlay and a local `HF_TOKEN`. Run `node --test tests/*.test.mjs` from `track_2b/` for the local suite. The benchmark CLI requires a real model endpoint and writes JSON and TSV results under `/artifacts`.

References

1. Xia et al., Agentless: Demystifying LLM-based Software Engineering Agents, 2024.
2. Zhang et al., AutoCodeRover: Autonomous Program Improvement, 2024.
3. Jimenez et al., SWE-bench, 2023; see also the SWE-bench Verified dataset and SWE-bench Pro.
4. Yang et al., SWE-smith, 2025; Zheng et al., SWE-Bench Pro Verified, 2026.
5. Lalonde et al., Revisiting Feedback-Driven LLM Code Repair, 2026.
6. Swiss AI Initiative, Apertus v1.5 model card and official serving instructions.
7. Hack Apertus, Track 2B project template and terms.
8. Gebert, Ponytail; Karpathy, autoresearch.
9. Cloudflare, workerd and Workers SDK.
10. Microsoft AI Agents Hackathon winners and Agent Academy event winners. RiskWise, ModelProof, and VendorGuard were reviewed as presentation examples, not described as prior Hack Apertus winners.

Report and authored documents: CC BY 4.0. Code: Apache-2.0. Dataset: CDLA-Permissive-2.0.